

Other Essential Functions

Other Essential Functions

There are a lot more functions defined in the `MPICH` implementation of `MPI`, most of which are *not* essential for most applications

We'll be going over some of the more *commonly used* ones, but for a more comprehensive list of functions

A good place to look is:

rookiehpc.org/mpi/docs/index.html

Barrier

```
1  int MPI_Barrier(MPI_Comm comm)
```

Creates a *synchronization point*. No process can pass this line of code until *all* processes in the communicator have reached it.

It's primarily used for *debugging*, *benchmarking*, and *input/output* processes

Note that `MPI_Barrier` kills parallel performance, so it should be used *sparingly* in production code.

Barrier Example

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <mpi.h>
4
5  int main() {
6      MPI_Init(NULL, NULL);
7
8      int my_rank;
9      MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
10
11     printf("[MPI process %d] I start waiting on the barrier.\n", my_rank);
12     MPI_Barrier(MPI_COMM_WORLD);
13     printf("[MPI process %d] I know all MPI processes have waited on the barrier.\n", my_rank);
14
15     MPI_Finalize();
16     return 0;
17 }
```

Isend and Irecv

Non blocking send and receive functions. They do the transfer and return **i**mmediately

This allows the program to do *other work* while the transfer is happening in the *background*.

```
1  int MPI_Isend(  
2      const void *buf,  
3      int count,  
4      MPI_Datatype datatype,  
5      int dest,  
6      int tag,  
7      MPI_Comm comm,  
8      MPI_Request *request  
9  )
```

```
1  int MPI_Irecv(  
2      void *buf,  
3      int count,  
4      MPI_Datatype datatype,  
5      int source,  
6      int tag,  
7      MPI_Comm comm,  
8      MPI_Request *request  
9  )
```

Isend example

```
1  int main() {
2      enum role_ranks { SENDER, RECEIVER };
3      switch(my_rank) {
4          case SENDER:
5              {
6                  int buffer_sent = 12345;
7                  MPI_Request request;
8                  MPI_Isend(&buffer_sent, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &request);
9                  MPI_Wait(&request, MPI_STATUS_IGNORE);
10                 break;
11             }
12         case RECEIVER:
13             {
14                 int received;
15                 MPI_Recv(&received, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
16                 break;
17             }
18         }
19     MPI_Finalize();
20     return 0;
21 }
```

Request

`MPI_Request` is a "*receipt*" for a background task, since `Isend` and `Irecv` return immediately, they don't return a status of the transfer.

So instead they return a request object that can be used to check on the status of the transfer later on.

```
1 MPI_Request request;  
2 MPI_Isend( ... , &request);
```

Request simply contains metadata about the transfer

It's almost always *passed* into an `MPI_Wait` or `MPI_Waitall` function to check on the status of the transfer

Wait and waitall

You *can't* safely read from or write to a buffer involved in a *non-blocking* transfer until the transfer is *complete*

`Wait` is simply a *synchronization point* for a *specific transfer*, it blocks until the transfer associated with the request is complete

Note that you can use `MPI_STATUS_IGNORE` if you don't care about the status of the transfer

```
1  int MPI_Wait(  
2      MPI_Request *request,  
3      MPI_Status *status  
4  )
```

Used for one request

```
1  int MPI_Waitall(  
2      int count,  
3      MPI_Request array_of_requests[],  
4      MPI_Status array_of_statuses[],  
5  )
```

Used for an array of requests, it blocks until *all* transfers associated with the requests are complete

Status

`MPI_Status` represents the status of a *receive* operation

Returned by receive operations (`MPI_Recv`), or non-blocking operation wait (`MPI_Wait` , `MPI_Waitall`).

In C the `MPI_Status` is a *structure* that contains **at least 3 attributes**:

- `MPI_SOURCE` ,
- `MPI_TAG` and
- `MPI_ERROR` .

When the user has no use of the `MPI_Status`, they can pass the constant `MPI_STATUS_IGNORE` (or `MPI_STATUSES_IGNORE` for array of statuses).

Status Example

```
1  int main() {
2      MPI_Init(NULL, NULL);
3      if(my_rank == 0) {
4          int buffer_sent = 12345;
5          int tag = 123;
6          MPI_Ssend(&buffer_sent, 1, MPI_INT, 1, tag, MPI_COMM_WORLD);
7      } else {
8          int buffer_received;
9          MPI_Status status;
10         MPI_Recv(&buffer_received, 1, MPI_INT,
11                 MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD,
12                 &status);
13         printf("%d, %d from rank %d, with tag %d and error code %d.\n",
14                my_rank,
15                buffer_received,
16                status.MPI_SOURCE,
17                status.MPI_TAG,
18                status.MPI_ERROR);
19     }
20     MPI_Finalize();
21 }
```

Sendrecv

`MPI_Sendrecv` is a combination of an `MPI_Send` and an `MPI_Recv`.

Which prevents *deadlock* in situations where two processes need to exchange data with each other.

```
1  int MPI_Sendrecv(  
2      const void *sendbuf,  
3      int sendcount, MPI_Datatype sendtype, int dest, int sendtag,  
4      void *recvbuf,  
5      int recvcount, MPI_Datatype recvtype, int source, int recvtag,  
6      MPI_Comm comm,  
7      MPI_Status * status  
8  )
```

Sendrecv Example

```
1  int main() {
2      MPI_Init(NULL, NULL);
3
4      int my_rank;
5      MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
6      int buffer_send = (my_rank == 0) ? 12345 : 67890;
7      int buffer_recv;
8      int tag_send = 0;
9      int tag_recv = tag_send;
10     int peer = (my_rank == 0) ? 1 : 0;
11
12     // Issue the send + receive at the same time
13     printf("MPI process %d sends value %d to MPI process %d.\n",
14           my_rank, buffer_send, peer);
15     MPI_Sendrecv(&buffer_send, 1, MPI_INT, peer, tag_send,
16                 &buffer_recv, 1, MPI_INT, peer, tag_recv, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
17     printf("MPI process %d received value %d from MPI process %d.\n",
18           my_rank, buffer_recv, peer);
19     MPI_Finalize();
20 }
```

Scatterv and gatherv

```
1  int MPI_Scatterv(  
2      const void *sendbuf,  
3      const int *sendcounts,  
4      const int *displs,  
5      MPI_Datatype sendtype,  
6      void *recvbuf,  
7      int recvcnt,  
8      MPI_Datatype recvtpe,  
9      int root,  
10     MPI_Comm comm  
11 )
```

A version of `MPI_Scatter` where the data dispatched from the root process can *vary* in the number of elements

```
1  int MPI_Gatherv(  
2      const void *sendbuf,  
3      int sendcount,  
4      MPI_Datatype sendtype,  
5      void *recvbuf,  
6      const int recvcounts[],  
7      const int displs[],  
8      MPI_Datatype recvtpe,  
9      int root,  
10     MPI_Comm comm  
11 )
```

A variant of `MPI_Gather` which allows the messages received to have **different lengths** and **be stored at arbitrary locations** in the root process buffer

Sendcounts and displacement

`sendcounts`, `recvcounts`, and `displacement` are arrays of size `world_size` that specify the number of elements sent to or received from each process

- `sendcounts[i]` specifies the number of elements sent to process `i` in `MPI_Scatterv`
- `recvcounts[i]` specifies the number of elements received from process `i` in `MPI_Gatherv`
- `displs[i]` specifies the displacement (offset) in the send buffer for process `i` in `MPI_Scatterv`, and in the receive buffer for process `i` in `MPI_Gatherv`

```
1  int sendcounts[world_size] = {2, 3, 1}; // example for 3 processes
2  int displs[world_size] = {0, 2, 5}; // example for 3 processes
3  MPI_Scatterv(..., sendcounts, displs, ...);
```

scatterv example

```
1  int main() {
2      MPI_Init(NULL, NULL);
3      int root_rank = 0;
4      int my_rank;
5      MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
6
7      switch(my_rank) {
8          case 0:
9              {
10                 int my_value;
11                 int buffer[7] = {100, 0, 101, 102, 0, 0, 103};
12                 int displacements[3] = {0, 2, 6};
13                 int counts[3] = {1, 2, 1};
14
15                 MPI_Scatterv(buffer, counts, displacements,
16                             MPI_INT, &my_value,
17                             1, MPI_INT,
18                             root_rank, MPI_COMM_WORLD);
19                 printf("Process %d received value %d.\n", my_rank, my_value);
20                 break;
21             }
```

scatterv example (cont)

```
1     case 1:
2     {
3         int my_values[2];
4         MPI_Scatterv(NULL, NULL, NULL,
5                     MPI_INT, my_values,
6                     2, MPI_INT,
7                     root_rank, MPI_COMM_WORLD);
8         printf("Process %d received values %d and %d.\n", my_rank, my_values[0], my_values[1]);
9         break;
10    }
11    case 2:
12    {
13        int my_value;
14        MPI_Scatterv(NULL, NULL, NULL,
15                    MPI_INT, &my_value,
16                    1, MPI_INT,
17                    root_rank, MPI_COMM_WORLD);
18        printf("Process %d received value %d.\n", my_rank, my_value);
19        break;
20    }
```

Gatherv example

```
1      int root_rank = 0;
2
3      switch(my_rank) {
4          case 0:
5              {
6                  int my_value = 100;
7                  int counts[3] = {1, 1, 2};
8                  int displacements[3] = {0, 3, 5};
9
10                 int* buffer = (int*)calloc(7, sizeof(int));
11                 MPI_Gatherv(&my_value, 1, MPI_INT,
12                             buffer, counts,
13                             displacements, MPI_INT,
14                             root_rank, MPI_COMM_WORLD);
15                 free(buffer);
16                 break;
17             }
```

Gatherv example

```
case 1:
{
    int my_value = 101;

    MPI_Gatherv(&my_value, 1, MPI_INT,
                NULL, NULL, NULL,
                MPI_INT, root_rank, MPI_COMM_WORLD);

    break;
}
case 2:
{
    int my_values[2] = {102, 103};

    MPI_Gatherv(my_values, 2, MPI_INT,
                NULL, NULL, NULL,
                MPI_INT, root_rank, MPI_COMM_WORLD);

    break;
}
```